FreeRTOS CheatSheet

FreeRTOS is a real-time operating system (RTOS) for embedded systems that provides multitasking and synchronization between tasks efficiently and lightweight.

Filter...

Start

Including FreeRTOS in a project

Include the main FreeRTOS header in your project:

```
#include "FreeRTOS.h"
#include "task.h"
```

Task Management

Task creation and management

Create a task

To create a task, `xTaskCreate` is used, providing the task function, name, stack size, parameters, and priorities.

```
xTaskCreate(
    vTaskCode,         // Task function
    "Task1",          // Task name
    1000,              // Stack size
    NULL,              // Parameters
    1,                 // Priority
    NULL               // Task handle (
);
```

Delete a task

To delete a task:

```
vTaskDelete(NULL);  // Deletes the cur
```

Suspend a task

To suspend a task:

```
vTaskSuspend(xTaskHandle);  // Suspend
```

Resume a suspended task

Resumes the execution of a suspended task.

```
vTaskResume(xTaskHandle);  // Resumes
```

Time control in tasks

Delay a task

To pause the task for a specified period of time

(in ticks):

```
vTaskDelay(pdMS_TO_TICKS(1000));  // P
```

Delay until the next tick

Adjusts the task to execute periodically:

```
vTaskDelayUntil(&xLastWakeTime, pdMS_T
```

Queues

Create a queue

Creates a queue to pass data between tasks.

```
xQueueHandle myQueue = xQueueCreate(10
```

Send data to a queue

Sends a value to the queue.

```
int value = 42;
xQueueSend(myQueue, &value, portMAX_DE
```

Receive data from a queue

Receives a value from the queue.

```
int receivedValue;
xQueueReceive(myQueue, &receivedValue,
```

Send from an interrupt

To send a message to a queue from an interrupt:

```
xQueueSendFromISR(myQueue, &value, &xH
```

Semaphores

Binary Semaphores

Create a binary semaphore

Used for task synchronization.

```
xSemaphoreHandle mySemaphore = xSemaph
```

Take a semaphore

Takes the semaphore to block execution until it is released.

```
xSemaphoreTake(mySemaphore, portMAX_DE
```

Give a semaphore

Releases the semaphore, allowing other tasks to take it.

```
xSemaphoreGive(mySemaphore);  // Relea
```

Mutex Semaphores

Create a Mutex

Mutex to prevent simultaneous access to shared resources.

```
xSemaphoreHandle myMutex = xSemaphoreC
```

Take and give a Mutex

Works the same as binary semaphores, but the Mutex must be taken and given by the same task.

```
xSemaphoreTake(myMutex, portMAX_DELAY)
xSemaphoreGive(myMutex);
```

Timers

Create a timer

Creates a timer that triggers a function after a period of time.

```
xTimerHandle myTimer = xTimerCreate(
    "Timer1",          // Name
    pdMS_TO_TICKS(1000),      // Period
    pdTRUE,                   // Repeat
    (void*)0,                 // Timer
    vTimerCallback            // Callba
);
```

Start a timer

Starts a timer.

```
xTimerStart(myTimer, 0);
```

Stop a timer

Stops a timer before it triggers.

```
xTimerStop(myTimer, 0);
```

Change a timer's period

Modifies the timer's period at runtime.

```
xTimerChangePeriod(myTimer, pdMS_TO_TI
```

DISCORD

Sections

- Courses
- Engineering
- Arduino
- ESP32
- Computing
- Development
- Raspberry Pi / Linux
- 3D printing
- Electronics
- Design

Interrupts

Configure interrupts

ISR (Interrupt Service Routine) functions can use FreeRTOS functions like `xQueueSendFromISR` or `xSemaphoreGiveFromISR` to interact with tasks.

Example of interrupt handling:

```
void ISR_Handler(void) {
    BaseType_t xHigherPriorityTaskWoke
    xSemaphoreGiveFromISR(mySemaphore,
    portYIELD_FROM_ISR(xHigherPriority
}
```

Memory Control

Check available memory

FreeRTOS includes a function to check the amount of free memory available in the heap.

```
size_t availableMemory = xPortGetFreeH
```

Define the stack size of a task

When creating a task, you can specify the stack size to be allocated:

```
xTaskCreate(vTaskCode, "Task", configM
```

On this page

- FreeRTOS CheatSheet
- Start
- Task Management
  - Task creation and management
  - Time control in tasks
- Queues
- Semaphores
  - Binary Semaphores
  - Mutex Semaphores
- Timers
- Interrupts
- Memory Control